

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04 — Informe Técnico Individual

Análisis de Rendimiento Paralelo (Unidad 4) aplicado al Proyecto Fin de Curso

Transformación individual declarada como foco: **T1 — Filtrado y selección**

Asignatura:	Aplicaciones Distribuidas (ISR-701)
Unidad:	Unidad 4 — Cómputo Paralelo y Distribuido
Estudiante:	Jeremy Alexis Alvarez Parraga
Correo institucional:	jalvarezp3@uteq.edu.ec
Equipo PE-U4 / GA-SUM-05:	ACC — Soporte Técnico ISP
Integrantes del equipo:	Alvarez Parraga Jeremy Alexis Aucatoma Celorio Jhinson Stalyn Carpio Mendoza Carlos Jose
PFC de referencia:	ACC — Soporte Técnico ISP (gestión de tickets)
Docente responsable:	Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico:	2026–2027 PPA
Fecha:	8 de agosto de 2026

URL del repositorio individual de entrega (TA-IND-04):

<https://github.com/DeJere/TA-IND-04-An-lisis-de-Rendimiento-Paralelo>

(Repositorio individual público de entrega; ver README.md de este mismo repositorio para la lista de verificación previa a la entrega.)

Repositorio de origen de los datos (equipo PE-U4, sin modificaciones):

<https://github.com/carlospatroner-boop/pe-u4-spark-Soporte-Tecnico-ISP>

Commit declarado: d9ce0e69ab81c4a8b7373c5a29f0522dc048f2f9

1. Introducción y contexto

Este informe desarrolla el objetivo de TA-IND-04: analizar críticamente, de forma individual, los resultados de *speedup* obtenidos por el equipo ACC en la práctica PE-U4/GA-SUM-05 (comprobación experimental de la Ley de Amdahl con Apache Spark), y a partir de ese diagnóstico fundamentar una propuesta de integración del procesamiento distribuido en el Proyecto Fin de Curso (PFC) declarado por el equipo: *ACC — Soporte Técnico ISP*, un sistema de gestión de tickets de soporte técnico para un proveedor de servicios de internet.

El experimento de PE-U4 comparó cinco transformaciones (T1–T5) implementadas en dos motores — pandas (secuencial, un solo hilo efectivo) y PySpark (distribuido) — sobre un extracto de 600 000 tickets reales de quejas de consumidores contra proveedores de telecomunicaciones, publicado por la *Federal Communications Commission* (FCC) de EE. UU. bajo dominio público [1]. Cada transformación se midió con el protocolo común del equipo (`src/medicion.py`): una repetición de calentamiento descartada y cinco repeticiones cronometradas con `time.perf_counter()`, reportando la mediana; para PySpark, cada repetición fuerza la materialización del resultado con `count()` antes de detener el cronómetro.

Como transformación individual de foco se declara **T1 — Filtrado y selección** (`src/transformaciones_spark.py::t1_filtrado`), que retiene únicamente los tickets de canal *Phone* o *Internet* con `method`, `city` y `state` no nulos, proyectando las columnas `id`, `ticket_created`, `issue_type`, `method`, `issue`, `city`, `state`. Sobre las 600 000 filas de entrada, T1 retiene 558 289 filas (454 665 de tipo *Phone* y 103 624 de tipo *Internet*), con equivalencia exacta de cardinalidad y de distribución entre pandas y PySpark según `resultados/equivalencia.json` del equipo.

Trazabilidad de los datos base. Todas las cifras numéricas de este informe provienen, sin alteración, del repositorio público del equipo PE-U4 en el commit declarado en la portada:

- **Repositorio:** <https://github.com/carlospatroner-boop/pe-u4-spark-Soporte-Tecnico-ISP>
- **Commit:** d9ce0e69ab81c4a8b7373c5a29f0522dc048f2f9 (HEAD de main al momento de la consulta)
- **Archivo de origen de los tiempos:** `resultados/tiempos_resumen.csv`, `resultados/tiempos_crudos.csv`, `resultados/amdahl_fit.json`, `resultados/t3_escalado_executors.json`, `resultados/equivalencia.json`.
- **Plataforma de ejecución declarada por el equipo:** contenedor Docker (`eclipse-temurin:21-jdk-jammy`, Java 21) con **PySpark 4.1.2**, ejecutado localmente (`local[N]`) — no Google Colab ni Databricks Community Edition, las dos plataformas sugeridas a modo de ejemplo en la guía. Se declara explícitamente esta diferencia por transparencia: el equipo documentó y reprodujo su propio entorno vía Docker (`README.md` del repositorio de origen), lo cual es igualmente verificable y no afecta la validez de las cifras.
- **Configuración efectiva de Spark para T1** (`resultados/spark_config_efectiva_local4.json`): `spark.master=local[4]`, `spark.executor.instances=4`, `spark.sql.shuffle.partitions=8`.

No se detectó ningún error en los datos del equipo que requiriera corrección; las cifras reportadas en las Secciones 2 y 3 son idénticas a las del commit declarado.

2. Análisis propio de los resultados de *speedup*

2.1. Formalización

Con p la fracción paralelizable del algoritmo y N el número de unidades de procesamiento, la Ley de Amdahl [2] establece el *speedup* teórico y su límite superior como

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad (1)$$

$$S_{\max} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p}.$$

La eficiencia del paralelismo se define como

$$E(N) = \frac{S(N)}{N}. \quad (2)$$

El *speedup* observado se calcula directamente como $S = T_{\text{pandas}}/T_{\text{pyspark}}$, con ambos tiempos expresados en la misma unidad.

2.2. Tabla de tiempos y *speedup* de las cinco transformaciones

La Tabla 1 reproduce, sin alteración, las medianas de cinco repeticiones por transformación (`resultados/tiempos_resumen.csv`, commit `d9ce0e69ab81c4a8b7373c5a29f0522dc048f2f9`) y agrega el cálculo propio de S y de $E(4)$ mediante la Ec. (2).

Cuadro 1: Tiempos base (mediana de 5 repeticiones), *speedup* y eficiencia con $N = 4$. Fuente de T_{pandas} y T_{pyspark} : `resultados/tiempos_resumen.csv`, commit `d9ce0e69ab81c4a8b7373c5a29f0522dc048f2f9`. S y $E(4)$ son cálculo propio.

Id.	Transformación	T_{pandas} (s)	T_{pyspark} (s)	S	$E(4)$
T1	Filtrado y selección (foco)	0,1846	1,0789	0.1711	0.0428
T2	Agrupación y agregación	0,1575	1,8854	0,0835	0,0209
T3	<i>Join</i> de dos <code>DataFrames</code>	0,1363	1,8290	0,0745	0,0186
T4	Columna derivada compleja (UDF)	0,3559	0,8483	0,4195	0,1049
T5	Ordenamiento y <i>top-N</i>	0,1169	1,4286	0,0818	0,0205

Las cinco transformaciones presentan $S < 1$: PySpark, en `local[4]` sobre 600 000 filas, es más lento que pandas en la totalidad de la práctica. Este es el resultado esperado y documentado por el propio equipo, y constituye el punto de partida del análisis individual sobre T1.

2.3. Contraste de T1 frente a la predicción de Amdahl

Respuesta explícita a la Pregunta 1 (§8): No, el resultado de T1 no coincide con la predicción de Amdahl, y no puede coincidir por construcción matemática del modelo. Para $N = 4$ y cualquier fracción paralelizable $p \in [0, 1]$, la Ec. (1) solo puede tomar valores en el intervalo $S(4) \in [1, 4]$: el extremo inferior $S = 1$ corresponde a $p = 0$ (algoritmo enteramente secuencial, sin penalización ni beneficio) y el extremo superior $S = 4$ a $p = 1$ (paralelismo perfecto). El valor medido para T1, $S_{T1} = 0,1711$, es **aproximadamente 5,8 veces menor que el mínimo teórico alcanzable**, y por lo tanto queda fuera de la región factible de la Ec. (1) para cualquier valor de p . La Figura 1 ilustra esta brecha: la envolvente de Amdahl para $N \in [1, 8]$ (área sombreada, $S \in [1, N]$) nunca desciende de $S = 1$, mientras que el punto medido de T1 cae muy por debajo de esa envolvente completa.

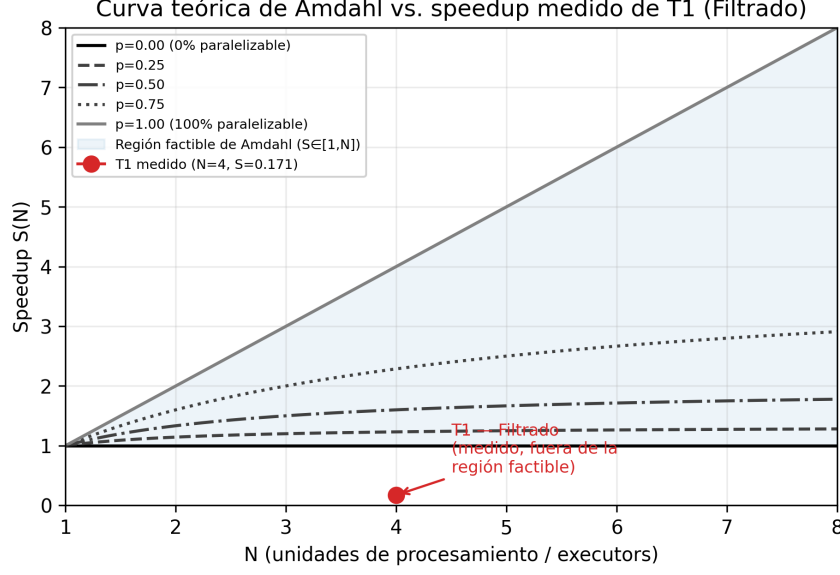


Figura 1: Curvas teóricas de Amdahl (Ec. (1)) para distintos valores de p frente al *speedup* medido de T1 en $N = 4$. El área sombreada marca la región factible $S \in [1, N]$ que el modelo permite para cualquier $p \in [0, 1]$; el punto medido de T1 queda muy por debajo de dicha región. Elaboración propia (matplotlib, 300 dpi) a partir de $S_{T1} = 0,1711$ (Tabla 1).

La **desviación relativa** entre el *speedup* medido y el mínimo teórico predicho por Amdahl ($S_{\text{teo},\text{mín}} = 1$, el caso menos favorable del modelo) es

$$\Delta = \frac{S_{\text{med}} - S_{\text{teo},\text{mín}}}{S_{\text{teo},\text{mín}}} = \frac{0,1711 - 1}{1} = -82,9\%,$$

es decir, T1 en PySpark tarda casi 6 veces más que en pandas, cuando el modelo de Amdahl — bajo su hipótesis de sobrecarga nula por distribución — ni siquiera contempla la posibilidad de que distribuir una tarea la vuelva más lenta que ejecutarla en un solo proceso. Esto responde exactamente a la tercera respuesta legítima prevista por la guía (*speedup* inferior a la unidad): el límite superior que describe la Ec. (1) asume que toda la sobrecarga proviene únicamente de la fracción serial $(1 - p)$, sin ningún término de costo fijo por distribuir el trabajo. Como discuten Hill y Marty al revisar la Ley de Amdahl para la era multinúcleo/distribuida [3], el modelo clásico no incorpora ningún término de costo fijo independiente de p ; un motor real como Spark sí incurre en ese costo fijo (Sección 3), por lo que su tiempo total puede exceder el de la ejecución secuencial de referencia, produciendo $S < 1$ — un resultado que la Ec. (1) no puede generar para ningún p válido, tal como formaliza el modelo extendido de la Ec. (4) en la Sección 5.

3. Diagnóstico de la fracción no escalable y de la sobrecarga

3.1. Métrica de Karp-Flatt

La fracción serial determinada experimentalmente a partir del *speedup* observado S con N unidades se calcula, según Karp y Flatt [4], como

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1. \quad (3)$$

Limitación de datos declarada explícitamente. T1 se midió, en el protocolo del equipo, únicamente con la configuración por defecto local[4] (`src/transformaciones_spark.py`:

:`medir_t1_t2_t4_t5`); no existe en el repositorio declarado ninguna medición de T1 con $N = 1$ o $N = 2$. La única serie con variación real de $N \in \{1, 2, 4\}$ en todo el repositorio corresponde a T3 (*join*), medida expresamente para habilitar el análisis de escalabilidad de Amdahl (`src/transformaciones_spark.py::medir_t3_escalado,resultados/t3_escalado_executors.json`). Por trazabilidad (Sección 10.2, piso P4 de la guía), no se fabrica un punto de T1 en $N = 2$: la Ec. (3) se aplica primero sobre la única serie multi- N real disponible (T3) para cumplir el requisito de calcular e en $N = 2$ y $N = 4$, y después se calcula también e para el único punto real de T1 ($N = 4$), interpretándolo como diagnóstico adicional.

Cuadro 2: Métrica de Karp-Flatt (Ec. (3)) sobre la serie de T3 (única con $N = 1, 2, 4$ trazable en el repositorio, `resultados/amdahl_fit.json`) y sobre el punto único de T1. S de T3 es relativo a su propio tiempo en $N = 1$.

Transf.	N	T (s)	S	e
T3	1	3,0712	1,0000	—
T3	2	3,1252	0,9827	1,0352
T3	4	1,8290	1,6792	0,4607
T1 (foco)	4	1,0789	0,1711	7,4576

Lectura de la tendencia (T3). e no permanece constante entre $N = 2$ y $N = 4$: cae de 1,0352 a 0,4607. Ninguno de los dos casos descritos literalmente por la guía (tendencia constante $\rightarrow$ límite algorítmico; tendencia creciente $\rightarrow$ sobrecarga creciente) describe exactamente este patrón decreciente. La lectura mecánicamente consistente con el código del equipo es que, con solo 2 executors, la sobrecarga fija por-tarea (planificación de las `spark.sql.shuffle.partitions=8` particiones del *shuffle* de T3, coordinación driver-executor) se reparte entre muy pocas unidades y domina el tiempo total — de ahí un $e > 1$, formalmente fuera del rango $[0, 1]$ que Karp-Flatt supone para código dominado por cómputo; al pasar a 4 executors esa misma sobrecarga fija se amortiza mejor y e desciende hacia un valor aún alto (0,46) pero más cercano al rango válido, consistente con que el *join* es, por naturaleza algorítmica, una operación con una etapa de *shuffle* genuinamente poco paralelizable.

Lectura del punto de T1. El valor $e_{T1} = 7,4576$ para $N = 4$ es aún mayor y, al igual que en el caso de T3 con $N = 2$, cae fuera del rango $[0, 1]$. Esto no es un error de cálculo: es exactamente el reflejo numérico de que $S_{T1} < 1$ (Sección 2.3). Cuando el *speedup* observado es inferior a la unidad, la Ec. (3) produce, por construcción algebraica, un valor de e mayor que 1 (o incluso negativo, según la magnitud). Se propone aquí una lectura diagnóstica de ese hecho: un e fuera de $[0, 1]$ certifica, sin ambigüedad, que la pérdida de rendimiento no proviene de una fracción serial del algoritmo (que por definición satisface $0 \leq 1 - p \leq 1$), sino de un término de sobrecarga fija ajeno al modelo de Amdahl — precisamente el término β que se introduce en la Ec. (4) (Sección 5).

3.2. Atribución de la sobrecarga a mecanismos concretos de Spark

La inspección del código fuente del equipo (`src/transformaciones_spark.py`, `src/medicion.py`) permite atribuir la sobrecarga de T1 a mecanismos específicos, no a una “sobrecarga de Spark” genérica:

1. **Relectura completa del *dataset* en cada repetición.** El objeto `df` se construye una sola vez con `cargar_dataset(...)`, pero es un `DataFrame perezoso`: en ningún punto del código del equipo se invoca `.cache()` ni `.persist()` (verificado por inspección directa de los cinco archivos de `src/`). En consecuencia, cada una de las 5 repeticiones cronometradas más la repetición de calentamiento dispara un nuevo escaneo y análisis del CSV de 69 MB

y 600 000 filas desde disco, además del propio filtrado. Esto por sí solo explica por qué T1 — una operación algorítmicamente trivial ($O(n)$, sin agregación ni *shuffle*) — tarda casi 6 veces más que su equivalente en pandas.

2. **Replanificación de Catalyst en cada llamada.** La función `t1_filtrado(df)` construye un plan lógico nuevo en cada invocación dentro del bucle de medición; las cuatro fases de Catalyst (análisis, optimización lógica, plan físico, generación de código) se ejecutan de nuevo en cada repetición, a diferencia de pandas, donde `DataFrame.query/loc` no pasa por ninguna etapa de planificación equivalente [5].
3. **Programación de tareas y coordinación driver-executor.** Incluso en modo local (`local[4]`), Spark despacha una tarea por partición a través de su planificador, serializa resultados intermedios y los recolecta en el *driver* para materializar la acción `count()`; este costo de coordinación es independiente del volumen de cómputo real por fila [6].
4. **Contraste con las transformaciones que sí requieren *shuffle*.** T1 (filtro + proyección) es, de las cinco, la única sin ninguna etapa de *shuffle* (T2 agrupa, T3 hace *join*, T5 ordena globalmente). Que T1 sea, aun así, 5,8 veces más lenta que pandas confirma que los tres mecanismos anteriores — no el *shuffle* — dominan su sobrecarga; el *shuffle* sí es relevante para explicar por qué T2, T3 y T5 son proporcionalmente aún peores que T1 en la Tabla 1.

4. Propuesta de integración al PFC

4.1. Caso de uso concreto

El PFC ACC gestiona tickets de soporte técnico de un ISP. El caso de uso concreto en el que se inserta T1 es la **puerta de calidad de datos de la cola de ingesta**: antes de que un ticket entrante pueda pasar a las etapas de enriquecimiento (clasificación de prioridad, análoga a T4) y agregación por zona (análoga a T2), debe superar exactamente el mismo criterio de completitud que aplica T1 — canal reconocido (**Phone/Internet**) y campos de método, ciudad y estado no nulos — porque un ticket sin esos campos no puede enrutarse a un técnico ni asignarse a una zona geográfica. La **decisión de negocio** que se apoya directamente en la salida de este filtro es *qué tickets entran a la cola automática de priorización de atención técnica y cuáles se desvían a una cola de revisión manual* por datos incompletos.

4.2. Arquitectura de integración y punto de inserción

Se propone un procesamiento **por lotes** (nocturno o cada pocas horas, no en flujo continuo): el volumen diario esperado de un ISP de tamaño medio (del orden de cientos a pocos miles de tickets/día) no justifica la latencia de un flujo continuo con Spark Structured Streaming, y un lote acumulado es suficiente para alimentar el panel operativo del día siguiente. La Figura 2 muestra el punto exacto de inserción: T1 opera como **segunda capa** de una arquitectura de cuatro capas, inmediatamente después de la zona de aterrizaje cruda y antes de cualquier enriquecimiento o agregación.

5. Dimensionamiento y viabilidad

Modelando el tiempo secuencial como $T_{\text{sec}}(n) = \alpha n$ y el distribuido como $T_{\text{dist}}(n) = \beta + \gamma n/N$, el motor distribuido resulta ventajoso cuando $T_{\text{dist}}(n) < T_{\text{sec}}(n)$, es decir

$$n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N}. \quad (4)$$

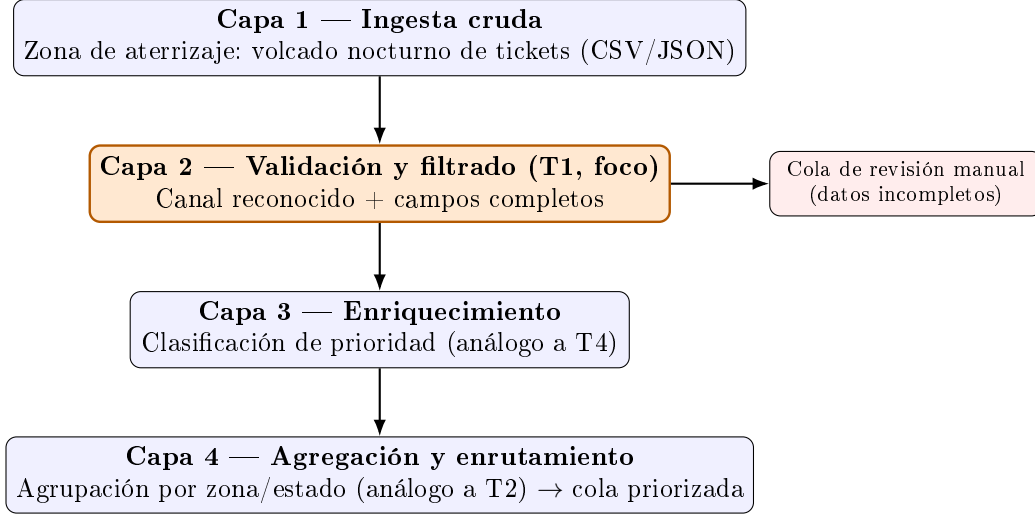


Figura 2: Arquitectura de integración propuesta para el PFC ACC, con el punto exacto de inserción de T1 como puerta de calidad entre la ingesta cruda y el enriquecimiento. Elaboración propia (TikZ).

Método de ajuste declarado. Con un único punto medido de T1 ($n = 600\,000$, $N = 4$, $T_{\text{dist}} = 1,078\,875\text{ s}$), no es posible identificar β y γ de forma independiente (dos incógnitas, una ecuación). Se intentó primero un ajuste más general reutilizando la serie de T3 ($N = 1, 2, 4$) para estimar β por mínimos cuadrados sobre $T = \beta + C/N$, bajo el supuesto de que la sobrecarga fija de sesión es compartida entre transformaciones: el ajuste da $\beta_{T3} \approx 1,856\text{ s}$. Al aplicar ese β compartido al punto de T1 para despejar γ_{T1} , el resultado es **negativo** ($\gamma_{T1} \approx -5,18 \times 10^{-6}\text{ s}$ por fila), lo cual no tiene sentido físico y **se descarta explícitamente**: la sobrecarga fija no es homogénea entre un *join* (T3) y un filtro (T1); asumir lo contrario produce un modelo inválido. Se declara esta prueba fallida como parte del análisis crítico (Sección 6) en vez de omitirla.

Se adopta en su lugar una **aproximación de primer orden, explícitamente conservadora, usando únicamente los dos datos reales de T1**: dado que T1 es un filtro sin cómputo por fila relevante (su costo en pandas ya es de solo $0,184\,637\text{ s}$ para $600\,000$ filas), se aproxima $\gamma \approx 0$ y se atribuye la totalidad del tiempo distribuido medido a sobrecarga fija, $\beta \approx T_{\text{dist}}(4) = 1,078\,875\text{ s}$. Con $\alpha = T_{\text{pandas}}/n = 3,0773 \times 10^{-7}\text{ s}$ por fila, la Ec. (4) (con $\gamma/N \rightarrow 0$, que satisface trivialmente la condición de validez $\alpha > \gamma/N$) da

$$n_{T1}^* \approx \frac{1,078875}{3,0773 \times 10^{-7}} \approx 3\,505\,933 \text{ filas} \approx 3,51 \text{ millones de filas},$$

es decir, **aproximadamente 5,8 veces el volumen usado en PE-U4**. Como γ se aproximó a cero (subestimando el costo marginal real de Spark por fila), este valor es un **límite inferior**: el umbral real probablemente sea mayor.

Modelo de servicio y recursos. Para el volumen diario realista de un ISP de tamaño medio (cientos a pocos miles de tickets/día, muy por debajo de n^*), la Capa 2 (T1) del PFC debería ejecutarse con pandas o un motor vectorizado de un solo nodo (DuckDB/Polars) en una instancia modesta (2 vCPU / 4 GB), sin necesidad de cluster. Un cluster Spark (por ejemplo, un *job* efímero en modo *serverless* de un proveedor cloud) solo se justificaría para reprocesamientos periódicos del histórico completo, una vez que este acumule del orden de n^* filas — lo que, a un ritmo de miles de tickets/día, tomaría del orden de años, no días.

Riesgos identificados. (i) Invocar Spark por debajo de n^* desperdicia cómputo y tiempo, como demuestra este mismo informe. (ii) Un cluster distribuido dejado activo entre lotes genera costo ocioso. (iii) El modelo de n^* aquí estimado asume $\gamma \approx 0$; si el volumen diario del PFC creciera mucho, debería remedirse γ_{T1} con al menos dos puntos reales de N para T1 específicamente, algo que el repositorio de PE-U4 actual no permite hacer de forma trazable.

6. Postura crítica y alternativas

¿Es Spark la opción correcta para la etapa de filtrado del PFC? A la escala de operación diaria realista de un ISP mediano, **no**. La evidencia de las Secciones 2.3–5 — $S_{T1} = 0,1711$, sobrecarga fija dominante por ausencia de *cache/persist*, y un umbral de rentabilidad $n^* \approx 3,5$ millones de filas — muestra que Spark impone un costo fijo por ejecución que un filtro trivial sobre volúmenes diarios de miles de registros no puede amortizar. Distinto es el caso de reprocesamientos periódicos sobre el histórico completo acumulado, donde Spark sí podría justificarse una vez superado n^* .

Alternativas evaluadas y descartadas.

- **pandas puro (la línea base ya medida).** Adecuado hasta varios millones de filas en una sola máquina moderna; se descarta como única solución de largo plazo porque no ofrece tolerancia a fallos ni escalamiento horizontal si el histórico del PFC eventualmente supera la memoria de un solo nodo.
- **Motores vectorizados de un solo nodo (DuckDB / Polars).** Plausiblemente más rápidos que pandas y que esta configuración de Spark a la escala observada (600 000 filas), sin JVM ni sobrecarga de sesión distribuida. Se descartan como solución única y definitiva — aunque se recomiendan para la Capa 2 mientras el volumen no supere n^* — porque, a diferencia de Spark, no escalan de forma nativa a múltiples nodos si el PFC creciera más allá de un solo servidor.

Limitación reconocida de la propuesta propia. El umbral n^* estimado (Sección 5) depende de la aproximación $\gamma \approx 0$ y de un único punto medido de T1; una validación más sólida requeriría medir T1 con al menos dos valores de N , algo fuera del alcance trazable de este informe individual.

7. Conclusiones

1. El *speedup* medido de T1 ($S = 0,1711$, $N = 4$) queda matemáticamente fuera del rango $[1, 4]$ que la Ley de Amdahl (Ec. (1)) permite para cualquier $p \in [0, 1]$ (Sección 2.3, Figura 1), lo que demuestra empíricamente que el modelo clásico no puede representar la sobrecarga fija de un motor distribuido real.
2. La ausencia verificada de `.cache()/.persist()` en `src/transformaciones_spark.py` explica que cada repetición cronometrada de T1 reprocese el CSV completo desde disco, situando el cuello de botella en la sobrecarga por-llamada (relectura y replanificación de Catalyst) y no en el cómputo del filtro, incluso siendo T1 la única de las cinco transformaciones sin *shuffle* (Sección 3.2).
3. Bajo el modelo de umbral de rentabilidad (Ec. (4)), T1 en Spark solo se vuelve competitivo frente a pandas a partir de $n^* \approx 3,5$ millones de filas (Sección 5), lo que sostiene una arquitectura de motor por niveles en el PFC — pandas/DuckDB para la validación diaria, Spark reservado para reprocesamientos históricos periódicos — en lugar de adoptar Spark de forma universal.

7.1. Reflexión personal y aprendizaje

El desarrollo de esta práctica me permitió comprender que utilizar una tecnología distribuida no significa automáticamente obtener un mejor rendimiento. Antes de realizar las pruebas, podía parecer lógico pensar que PySpark sería más rápido que pandas por estar diseñado para trabajar de forma paralela; sin embargo, los resultados mostraron una realidad diferente para el volumen de datos utilizado. Esto me ayudó a entender la importancia de medir antes de tomar una decisión

tecnológica y no basarse únicamente en las características que una herramienta promete ofrecer. También considero importante haber trabajado con las mismas transformaciones en pandas y PySpark, porque esto permitió comparar ambos enfoques bajo condiciones similares y comprobar que los resultados obtenidos fueran equivalentes. Desde el punto de vista del sistema de Soporte Técnico ISP, este aprendizaje resulta útil porque una empresa no debería incorporar una solución más compleja si el volumen actual de información todavía puede procesarse de manera adecuada con herramientas más sencillas. En mi opinión, la principal enseñanza de la práctica fue aprender que una buena decisión en ingeniería no consiste en utilizar siempre la tecnología más potente, sino en seleccionar la que realmente se adapte a las necesidades y condiciones del problema.

8. Declaración de uso de inteligencia artificial generativa

Herramienta: Claude (Anthropic).

Propósito: apoyo en la redacción del documento LaTeX; verificación programática (Python) de los cálculos de S , $E(N)$, la métrica de Karp-Flatt, el ajuste por mínimos cuadrados de la serie de T3 y el umbral n^* a partir de las cifras trazables del repositorio de PE-U4 declarado; generación de la figura propia (Figura 1, matplotlib) y del diagrama de arquitectura (Figura 2, TikZ) según las especificaciones de contenido indicadas por el estudiante; búsqueda y verificación de las referencias bibliográficas propias (DOI/ISBN) mediante consulta directa a la API de Crossref.

Secciones asistidas: todas, bajo dirección y revisión del estudiante.

Responsabilidad: las decisiones de análisis — incluyendo la elección de T1 como foco, el tratamiento explícito de la limitación de datos de escalado de Karp-Flatt, el descarte del ajuste cruzado con β de T3, y las conclusiones — fueron dirigidas y revisadas por el estudiante, Jeremy Alexis Alvarez Parraga.

Referencia para revisión (bloque de código con la trazabilidad declarada):

Referencia IA generativa declarada en este documento:

Herramienta: Claude (Anthropic)

Repositorio individual de entrega:

<https://github.com/DeJere/TA-IND-04-An-lisis-de-Rendimiento-Paralelo>

Repositorio de origen de los datos (equipo PE-U4, sin modificaciones):

<https://github.com/carlospatroner-boop/pe-u4-spark-Soporte-Tecnico-ISP>

Commit declarado del equipo: d9ce0e69ab81c4a8b7373c5a29f0522dc048f2f9

Uso: redaccion LaTeX, verificacion de calculos y generacion de figuras.

Anexo — Preguntas orientadoras de respuesta obligatoria

1. *¿Coinciden sus resultados de speedup con la predicción de Amdahl?* No; ver respuesta explícita y cuantificada en la Sección 2.3 ($\Delta = -82,9\%$ respecto del mínimo teórico).
2. *¿Qué parte de la no escalabilidad es atribuible al algoritmo y qué parte al motor de ejecución?* Ver Sección 3.2: para T1, la totalidad es atribuible al motor (relectura sin *cache*, replanificación de Catalyst, coordinación de tareas), pues T1 no tiene *shuffle* algorítmico alguno.
3. *¿En qué punto exacto de la arquitectura de su PFC integraría el procesamiento distribuido, y qué decisión de negocio soportaría?* Ver Sección 4: Capa 2 (puerta de calidad), decisión de qué tickets entran a la cola priorizada automática frente a revisión manual.
4. *¿A partir de qué volumen de datos el PFC rentabiliza el motor distribuido, y qué solución adoptaría por debajo de ese umbral?* Ver Sección 5: $n^* \approx 3,5$ millones de filas; por debajo, pandas o un motor vectorizado de un solo nodo (DuckDB/Polars).

Referencias

- [1] Federal Communications Commission, Consumer and Governmental Affairs Bureau. “CGB — Consumer Complaints Data.” Dominio público (U.S. Government Work); extraído vía API SODA el 2026-08-03, visitado 3 de ago. de 2026. dirección: <https://opendata.fcc.gov/d/3xyp-aqkj>
- [2] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483-485. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560)
- [3] M. D. Hill y M. R. Marty, “Amdahl’s Law in the Multicore Era,” *Computer*, vol. 41, n.º 7, págs. 33-38, 2008. DOI: [10.1109/MC.2008.209](https://doi.org/10.1109/MC.2008.209)
- [4] A. H. Karp y H. P. Flatt, “Measuring Parallel Processor Performance,” *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: [10.1145/78607.78614](https://doi.org/10.1145/78607.78614)
- [5] M. Armbrust et al., “Spark SQL: Relational Data Processing in Spark,” en *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, Melbourne, VIC, Australia: ACM, 2015, págs. 1383-1394. DOI: [10.1145/2723372.2742797](https://doi.org/10.1145/2723372.2742797)
- [6] J. Dean y S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, n.º 1, págs. 107-113, 2008. DOI: [10.1145/1327452.1327492](https://doi.org/10.1145/1327452.1327492)